

Technology Stack

Date	04 July 2026
Team ID	ramanharshitha2005@gmail.com
Project Name	OptiCrop - Smart Agricultural Production Optimization Engine
Maximum Marks	2 Marks

Define Your Technology Stack:

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side	HTML5, CSS3, JavaScript (Vanilla), Chart.js 4.4.1	Five semantic HTML pages (index, recommend, disease, insights, contact) with a single shared styles.css. Vanilla JS handles all API calls via fetch(). Chart.js renders bar, radar, and scatter charts on the analytics dashboard. No build step or bundler needed — runs directly in any modern browser.
2	Backend / Server-Side	Python 3.10+, http.server, BaseHTTPRequestHandler, ThreadingHTTPServer	Pure Python HTTP server with no external web framework dependency. ThreadingHTTPServer handles concurrent requests. All API routes (/api/recommend, /api/suitability, /api/fertilizer, /api/disease, /api/image-disease, /api/weather, /api/analytics) are implemented in a single server.py file.
3	Machine Learning	Scikit-learn 1.4+ (RandomForestClassifier, DecisionTreeClassifier, SVC, LabelEncoder, StandardScaler, Pipeline), XGBoost (optional), Joblib	Crop recommendation uses an ensemble of 3 models (RF, DT, SVC) with probability averaging — trained in ml/train_model.py and saved as crop_model.joblib. Disease classification uses a separate Random Forest (disease_model.joblib) trained in ml/train_disease_model.py. Image disease uses a Keras CNN (optional, image_disease_model.joblib or .h5).
4	Deep Learning (Image)	TensorFlow-CPU 2.14+, Keras, Pillow (PIL)	Optional CNN model for image-based plant disease identification. Images are resized to 128x128 or 224x224 px, normalised, and classified using a pre-trained or custom-trained Keras model. Pillow handles image opening and pixel-level feature extraction for the lightweight joblib-based fallback model.
5	Data / Storage	CSV (Crop_recommendation.csv — 2200 rows, 8 columns), JSON (opticrop-data.json — disease profiles and fallback crop data), Joblib model files	No database used. Crop data is loaded from CSV using Pandas at runtime. The opticrop-data.json provides fallback crop profiles and disease records. Trained model artifacts (crop_model.joblib, disease_model.joblib) are stored in the models/ folder.

6	Cloud / Hosting / Deployment	Local machine only — http://127.0.0.1:8000	No external hosting in the current version. The app is launched by running: <code>python3 server.py</code> . It binds to 0.0.0.0:8000 making it accessible on the local network (e.g., 192.168.x.x:8000 on LAN). Future deployment could use Render, Railway, or a VPS.
7	Version Control & CI/CD	Git, GitHub (https://github.com/RamanHarshitha/Smart_Bridge)	Full project source code is version-controlled on GitHub. Commit history tracks model training iterations, <code>server.py</code> updates, and frontend changes. No CI/CD pipeline configured at this stage.
8	Third-Party APIs / Other Tools	Open-Meteo Geocoding API, Open-Meteo Forecast API, Pandas, NumPy, Matplotlib, Seaborn	Open-Meteo is a free, open-source weather API requiring no API key — used for real-time temperature, humidity, and 30-day rainfall. Pandas and NumPy are used for dataset loading and ML feature engineering. Matplotlib and Seaborn generate confusion matrix plots during model training.